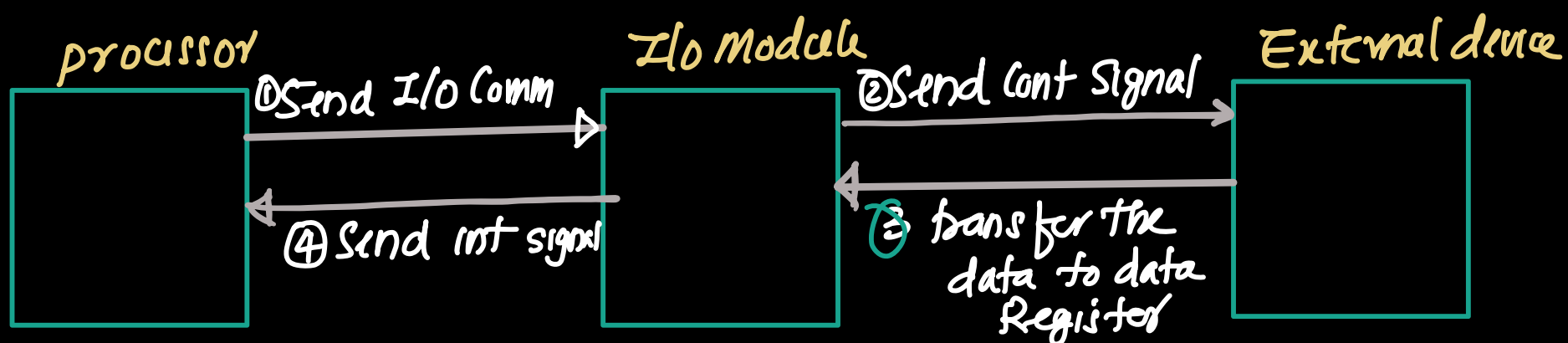
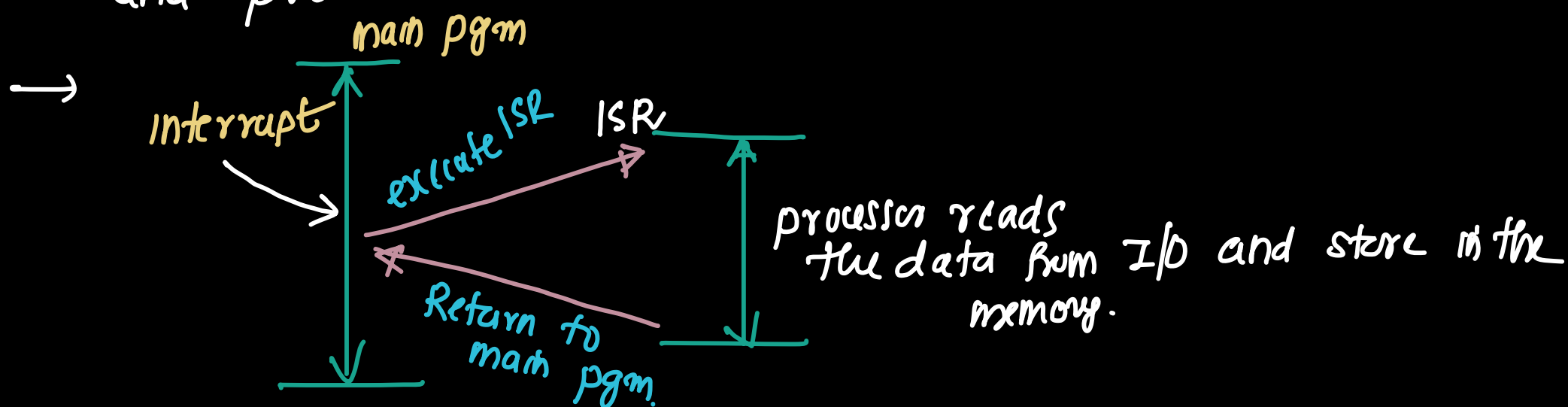


Interrupt driven I/O (Note)

- In programmed I/O, processor must continuously check the status of the I/O module to determine whether the device is ready to transfer the data.
- thus processor can't do other useful work. which result in a wasting processor time and degrading system performance.
- To overcome this, interrupt driven I/O is used.
- In Interrupt driven I/O, processor issues an I/O command to the I/O module. and processor continue the execution of other instruction.
- On receiving I/O command, I/O Module read the data from corresponding peripheral device and store the data in data register of I/O module.



- When data is available in the I/O module, I/O module sends an interrupt signal to the processor.
- processor complete the execution of current instruction and process the interrupt.



→ Processor execute the ISR . During the ISR execution (N)
processor reads the data from the I/O device and store
the data in memory. once ISR is completed processor resumes
the main pgm.

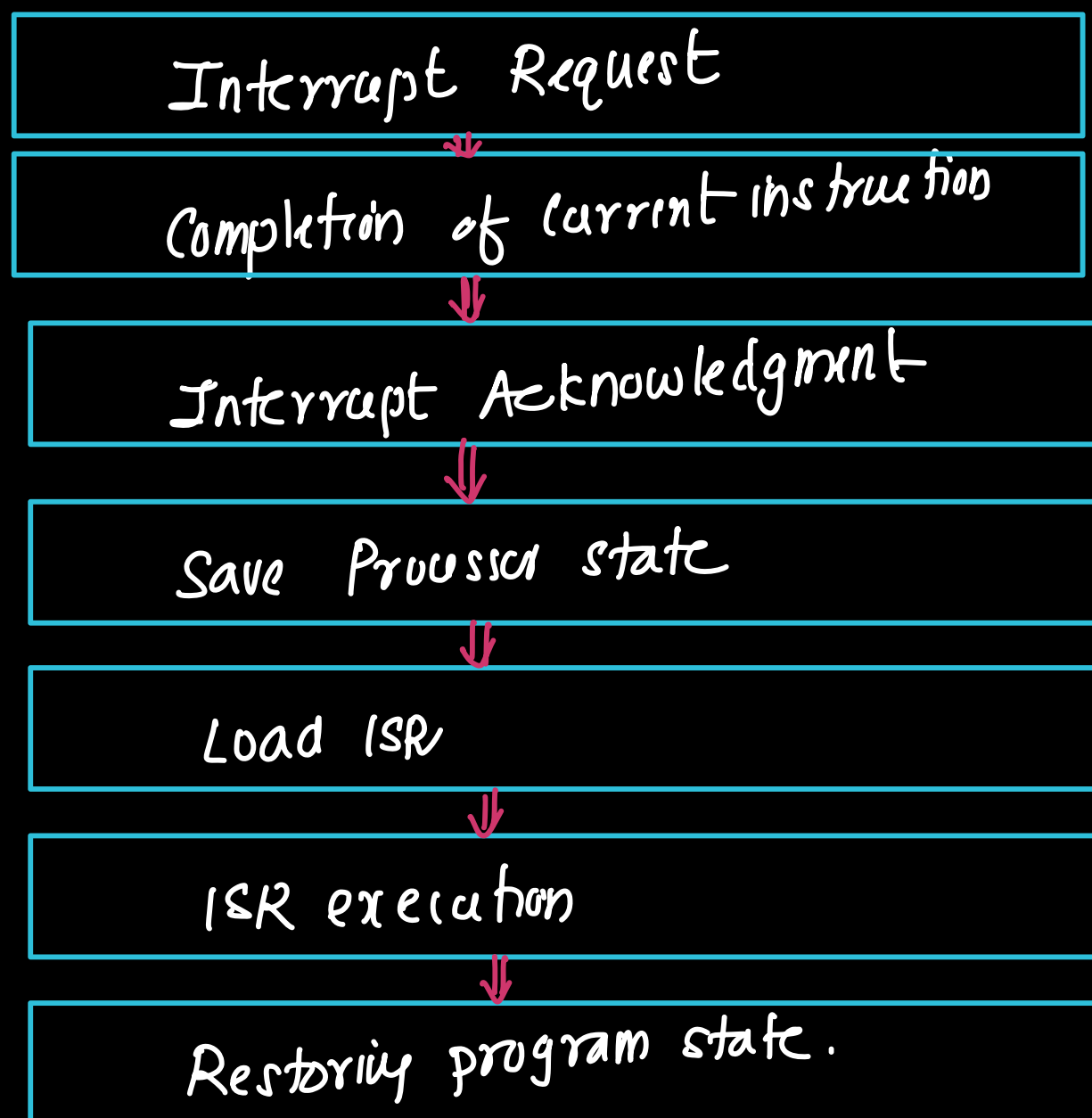
Operation of interrupt driven I/O (Note)

- processor send a read command to the I/O module
- The I/O module read data from associated peripheral device
- The data stored in the data Register of the I/O module.
- when the data becomes available, the I/O module sends an interrupt signal to the processor indicating that it require service (**Interrupt Request**)
- processor complete the execution of current instruction before responding to the interrupt. (**completion of current instruction**)
- processor checks for interrupts, determine, interrupt has occurred, and send acknowledgment signal to the device. This acknowledgment allows the device to remove its interrupt signal. (**Interrupt Acknowledgment**)
- Processor saves information required to resume the main program.
 - Address of next instruction (content of PC) and
 - status information (content of PSW)then values are pushed to the stack. (**Saving processor state**)
- processor transfer control to execute interrupt service routine. the processor loads the Add of ISR to the PC (**loading interrupt Service Routine**)
- During ISR execution processor read the status Register of the I/O module. Depending on the operation
 - ① input operation
The processor read the data from data Register of the I/O module
 - ② Output operation
processor write few data into data Register (**ISR execution**)

- (2)
- After all required actions are completed, content of PC and PSW are restored and processor returns to the main program and continue the execution from where it was stopped. (Restore program status)

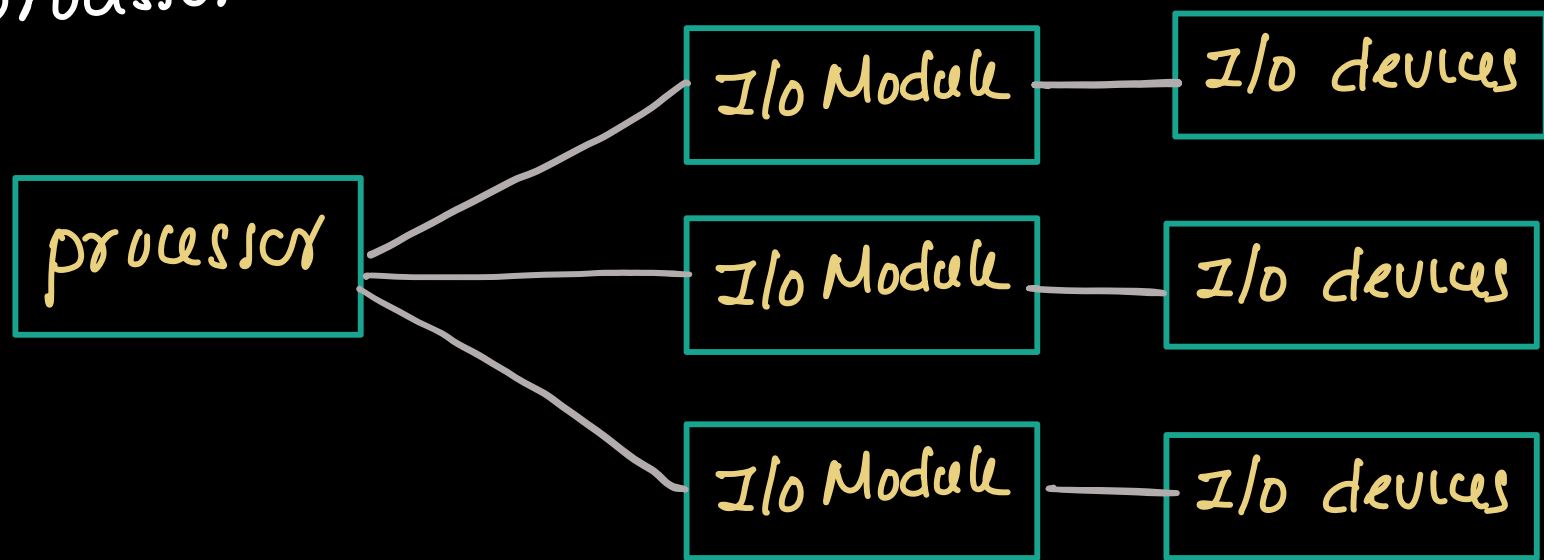
Interrupt processing steps.

- ① Interrupt Request
- ② Completion of current instruction
- ③ Interrupt Acknowledgment
- ④ Save Processor state
- ⑤ Load ISR
- ⑥ ISR execution
- ⑦ Restoring program state.



Design issues in Interrupt driven I/O (Note)

- In a computer system, multiple I/O devices are connected to the processor



- When interrupt driven I/O is used, the device may generate interrupt request to the processor.

Design issues

- When interrupt occurs, processor must determine which I/O device / I/O module generated the request.
- If multiple interrupt occurs at same time, processor must decide which interrupt should be serviced first.

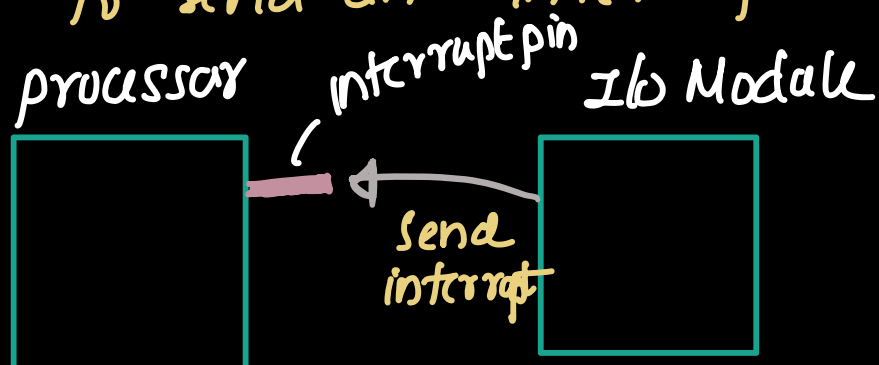
The first issue (identify the device which request interrupt) can be solved using following techniques.

- ① Multiple interrupt line
- ② Software polling
- ③ Daisy chain

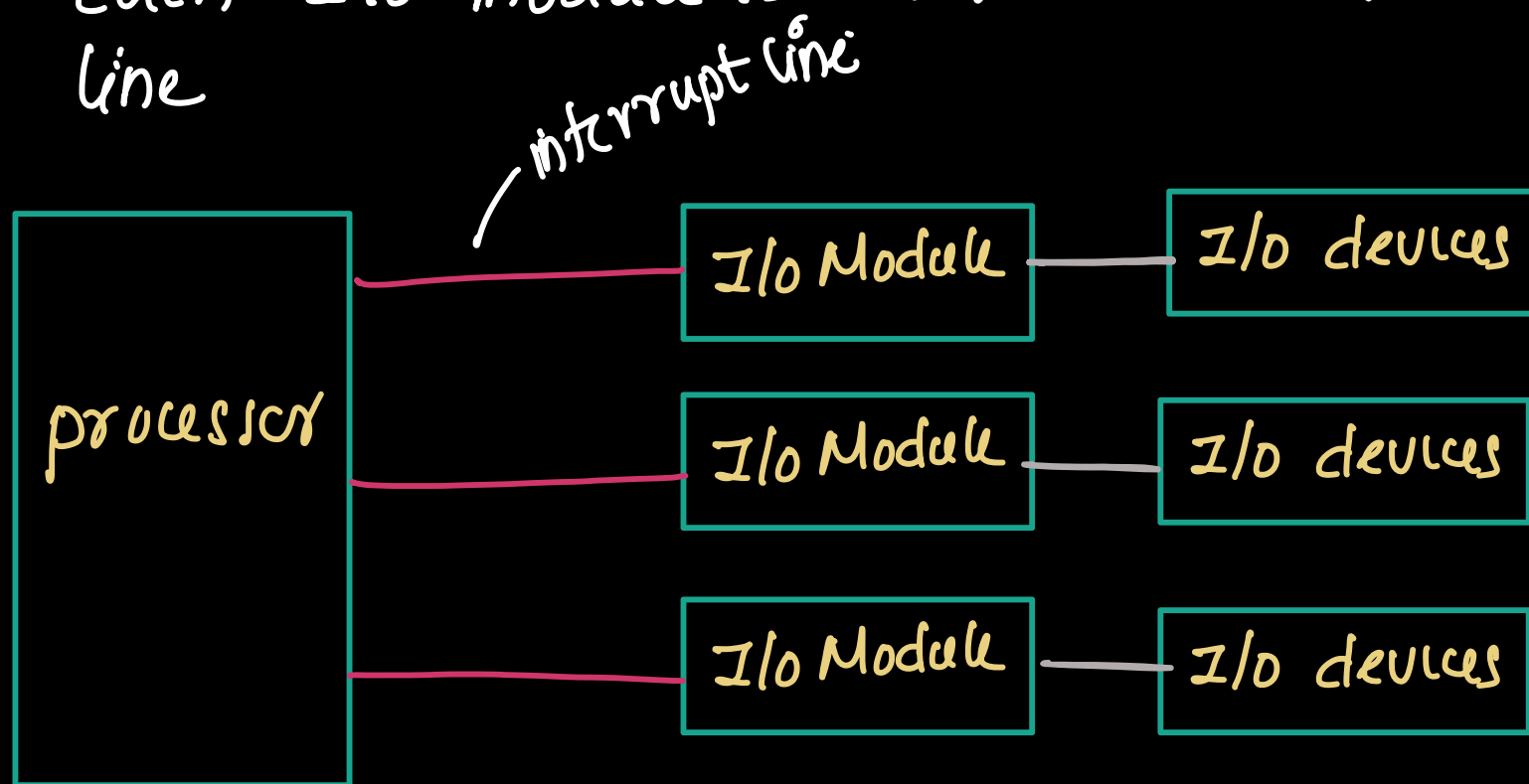
① Multiple interrupt line.

②

- normally processor have dedicated pin to allow an I/O device to send an interrupt signal



- To identify the device who generated interrupt, multiple interrupt lines are provided
- Each I/O module is connected to separate interrupt line



→ when an device / (I/O) module need the service from the processor, it raises its own interrupt line. thus processor can immediately identify which device has generated the interrupt

Disadvantage

- ① It is impractical to provide the large no. of interrupt lines because the processor pins are limited

② Software Polling

- When an interrupt occurs, processor execute and interrupt service routine that check each I/O module one by one.
- When processor detect an interrupt, processor executing a polling routine. processor check the status register of each I/O module one by one.
- When the module that generated the interrupt is found. processor branch to that device specific ISR.

disadvantage.

- processor check multiple devices sequentially (one by one) so this technique is time consuming.

③ Daisy chain

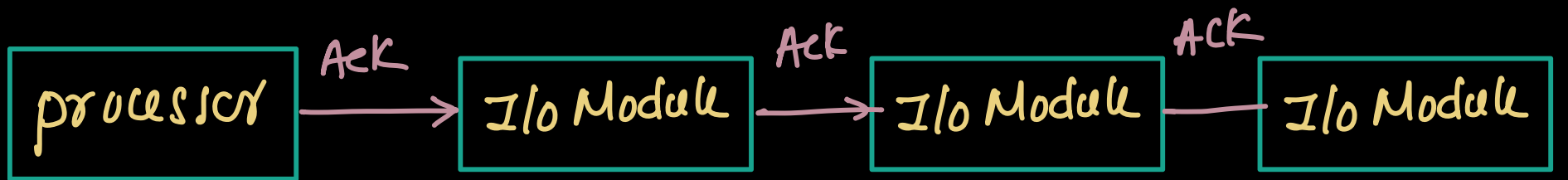
- In this technique, all I/O modules share a common interrupt line connected to the processor.

— Operation

- when an I/O device require service, it sends an interrupt signal through its I/O module to the processor using common interrupt signal



- Once processor detect the interrupt, processor send an acknowledge signal. this signal indicate that processor is ready to determine which device generated the interrupt.
- this ack signal travels through the I/O modules one by one in sequence. this arrangement forms a daisy chain.



— each module perform the following operation

- ① if it did not generate the interrupt, it passes ack signal to next I/O module in the chain
- ② if it generated the interrupt, then placing a word on data line. this word is called vector. this represent the address of I/O module

— processor reads the vector from the data bus and use it to determine which ISR to execute.

- Explain the operation of Interrupt-Driven I/O.
- Describe the sequence of operations in interrupt processing.
- Explain interrupt processing with the help of a diagram.
- Explain the steps performed by the processor when an interrupt occurs.
- Explain the working of Interrupt-Driven I/O from the point of view of the processor and the I/O module.
- What are the design issues in interrupt-driven I/O?
- Explain different methods used for identifying the interrupting device.
- Explain the following interrupt identification techniques:
 - Multiple interrupt lines
 - Software polling
 - Daisy chain